

IUT — Département Réseaux & Télécommunications
BUT RT — 2ème année

CinéBase

Application web de gestion d'une banque de films

SAÉ RT3 — Développement d'une application web

Réalisé par :

Alessandro · Ibrahim · Mathis

Année universitaire 2025 – 2026

Sommaire

1. Introduction
2. Présentation du projet
3. Analyse des besoins
4. Conception de la base de données
5. Architecture de l'application
6. Fonctionnalités développées
7. Organisation du travail
8. Difficultés rencontrées
9. Conclusion

1. Introduction

Dans le cadre de la SAÉ RT3, notre groupe a été chargé de concevoir et de développer une application web complète de gestion d'une banque de films. Ce projet nous a permis de mobiliser des compétences en développement web, en conception de bases de données relationnelles, ainsi qu'en déploiement sur serveur Linux.

L'application, nommée **CinéBase**, permet à des utilisateurs de consulter des films, de les noter et de les commenter. Elle distingue deux types d'utilisateurs — les professionnels et les amateurs — afin de proposer des statistiques différenciées.

2. Présentation du projet

CinéBase est une application web développée avec le framework **Django** (Python). Elle offre une interface complète pour gérer une base de films, d'acteurs, de catégories et de

commentaires, accessible depuis n'importe quel navigateur.

2.1 Objectifs

- Permettre la gestion complète (ajout, modification, suppression) des films, acteurs et catégories
- Proposer un système d'inscription et de connexion avec deux types de comptes
- Calculer des moyennes de notes séparées selon le type d'utilisateur
- Permettre l'import de films en masse via un fichier CSV
- Déployer l'application sur un serveur Linux

2.2 Technologies utilisées

Technologie	Version	Rôle
Python	3.13	Langage de programmation
Django	4.2+	Framework web back-end
SQLite	—	Base de données
Bootstrap	5.3.2	Interface graphique responsive
Bootstrap Icons	1.11.3	Icônes
Apache / Gunicorn	—	Serveur web (déploiement)
Git / GitHub	—	Gestion de versions

3. Analyse des besoins

3.1 Besoins fonctionnels

- Gestion des films (titre, année, réalisateur, affiche, catégorie)
- Gestion des acteurs et du casting (liaison film ↔ acteur)
- Gestion des catégories
- Système d'authentification (inscription, connexion, déconnexion)
- Commentaires et notes (1 à 10) par film et par utilisateur
- Calcul de la note moyenne globale, professionnelle et amateur

- Mise en valeur du meilleur et du pire commentaire
- Import de films par fichier CSV

3.2 Besoins non fonctionnels

- Interface responsive (compatible mobile et desktop)
- Application sécurisée (authentification requise pour les actions d'écriture)
- Déploiement sur serveur Linux avec serveur web

4. Conception de la base de données

4.1 Modèle relationnel

La base de données est composée de 6 tables :

Table	Description	Clé primaire
categorie	Catégories de films	id_categorie
film	Films avec leurs informations	id_film
acteur	Acteurs	id_acteur
film_acteur	Table de liaison film ↔ acteur (casting)	id_film + id_acteur
personne	Utilisateurs (professionnel/amateur)	id_personne
commentaire	Notes et commentaires sur les films	id_commentaire

4.2 Relations

- Un film appartient à **une seule catégorie**
- Un film peut avoir **plusieurs acteurs** (via film_acteur)
- Un acteur peut jouer dans **plusieurs films**
- Un utilisateur peut déposer **un commentaire par film**
- Un film peut avoir **plusieurs commentaires**

5. Architecture de l'application

5.1 Structure Django (MVT)

Django suit le patron de conception **MVT** (Modèle – Vue – Template) :

- **Modèle** : définit la structure des données (`models.py`)
- **Vue** : contient la logique métier (`views.py`)
- **Template** : gère l'affichage HTML (`templates/`)

5.2 Structure des fichiers

```
banque_films/  
├── banque_films/          # Configuration du projet  
│   ├── settings.py        # Paramètres Django  
│   └── urls.py            # Routes principales  
├── films/                 # Application principale  
│   ├── models.py          # Modèles de données  
│   ├── views.py           # Vues (logique)  
│   ├── urls.py            # Routes de l'application  
│   ├── forms.py           # Formulaires  
│   ├── admin.py           # Interface d'administration  
│   └── templates/         # Pages HTML  
├── static/                # CSS, images  
├── manage.py  
└── requirements.txt
```

5.3 Modèle utilisateur personnalisé

Django impose par défaut un modèle utilisateur standard. Pour ajouter le champ `type` (professionnel/amateur), nous avons créé un modèle `Personne` qui hérite de `AbstractUser` et l'avons déclaré dans les paramètres via `AUTH_USER_MODEL = 'films.Personne'`.

6. Fonctionnalités développées

6.1 Authentification

L'application propose un système complet d'inscription et de connexion. Lors de l'inscription, l'utilisateur choisit son type de compte : **amateur** ou **professionnel**. Ce type est utilisé pour calculer des moyennes de notes différenciées.

6.2 Gestion des films

La page de liste des films propose une **recherche par titre** et un **filtre par catégorie**. Chaque film dispose d'une fiche détaillée affichant :

- La note moyenne globale
- La note moyenne des professionnels séparément
- La note moyenne des amateurs séparément
- Le meilleur et le pire commentaire mis en évidence
- La liste des acteurs du casting

6.3 Import CSV

Un formulaire d'import permet d'ajouter plusieurs films en une seule opération via un fichier CSV. Le format attendu utilise le point-virgule comme séparateur de colonnes et le pipe (|) pour séparer plusieurs acteurs.

```
titre;annee_sortie;realisateur;categorie;acteurs
Inception;2010;Christopher Nolan;Science-Fiction;Leonardo DiCaprio|Joseph Gordon-
```

6.4 Interface d'administration

Django fournit une interface d'administration accessible à l'adresse `/admin`. Tous les modèles y sont enregistrés, permettant une gestion complète de la base de données sans passer par l'interface utilisateur.

7. Organisation du travail

7.1 Répartition des tâches

Membre	Tâches réalisées
Alessandro	Développement de l'application Django complète, modèles, vues, templates, formulaires, import CSV, organisation du dépôt GitHub
Ibrahim	Déploiement sur VM Linux, configuration Apache/Nginx, mise en production de l'application

Mathis

Rédaction du rapport, schéma relationnel, script SQL de création de la base de données, tests de l'application

7.2 Gestion de versions — GitHub

Le code source est hébergé sur GitHub et organisé en branches :

- `main` — branche principale, contient le schéma SQL
- `alessandro` — code source de l'application Django
- `ibrahim` — configuration du serveur Linux
- `mathis` — rapport et schéma relationnel

À la fin du projet, toutes les branches sont fusionnées dans `main`.

8. Difficultés rencontrées

8.1 Modèle utilisateur personnalisé

L'intégration d'un modèle utilisateur personnalisé dans Django nécessite de le déclarer avant la première migration. Cette contrainte nous a obligés à bien planifier la structure du projet dès le départ pour éviter des conflits de migrations.

8.2 Import CSV avec encodage

Les fichiers CSV créés sous Windows contiennent parfois un BOM (Byte Order Mark) en début de fichier. Nous avons résolu ce problème en utilisant l'encodage `utf-8-sig` lors de la lecture du fichier, qui gère ce cas automatiquement.

8.3 Calcul des moyennes par type d'utilisateur

Le calcul des moyennes différenciées (professionnels vs amateurs) a nécessité d'écrire des méthodes personnalisées sur le modèle `Film`, en filtrant les commentaires selon le type de la personne associée.

9. Conclusion

Ce projet nous a permis de développer une application web complète, de la conception de la base de données jusqu'au déploiement sur serveur Linux. Nous avons pu mettre en pratique les compétences acquises en cours : développement back-end avec Django, conception de bases de données relationnelles, administration système Linux et gestion de versions avec Git.

L'application CinéBase répond à l'ensemble des besoins fonctionnels définis au départ : gestion complète des films, acteurs et catégories, système de notes différencié par type d'utilisateur, import CSV, et interface responsive accessible depuis n'importe quel navigateur.

Ce projet nous a également permis de travailler en équipe avec une organisation claire des tâches et une gestion du code via GitHub, compétences essentielles dans un environnement professionnel.